

APPENDICE – Servizi REST

Introduzione ai Servizi REST

Vediamo quali strumenti ci dà Spring per esporre un controller in versione REST. E la risposta del docente è, testualmente, che *"in Spring è semplicissimo"*: basta cambiare un'annotazione.

@Controller vs @RestController. Nei video precedenti abbiamo sempre lavorato con `@Controller`, cioè con Spring MVC classico, dove il metodo ritorna il **nome di una vista** che poi viene risolto in una pagina HTML. Per i servizi REST non si usa `@Controller` ma **@RestController**, e da quel momento **tutti i metodi della classe vengono automaticamente visti come servizi REST**: il valore ritornato non è più il nome di una vista, ma diventa direttamente il **corpo della risposta HTTP**, serializzato in JSON.

L'esempio che mostra è un metodo esposto su `/employee-rest/prova`, che `produces = "application/json"` e restituisce un'istanza della classe `Employee` (composta da `nome` e `email`). Quando lo richiama dal browser, quello che appare non è una pagina ma il JSON dell'oggetto:

```
@RestController
@RequestMapping("/employee-rest")
public class EmployeeRestController {

    @RequestMapping(value = "/prova", produces = "application/json")
    public Employee prova() {
        Employee e = new Employee();
        e.setNome("Mario");
        e.setEmail("mario@rossi.it");
        return e;
    }
}
```

La conversione dell'oggetto Java in JSON avviene **dietro le quinte**: non c'è nessuna riga di codice che serializza, ci pensa Spring (tramite Jackson) perché la classe è annotata `@RestController`.

Passare parametri come parte dell'URL: @PathVariable. Questo è il "piccolo appunto che non abbiamo detto" della lezione. È possibile mettere una **parentesi graffa con un nome qualsiasi** dentro l'URL, e legarla a un parametro del metodo con `@PathVariable`:

```
@RequestMapping("/{name}")
public Employee get(@PathVariable String name) {
```

```
Employee e = new Employee();
e.setNome(name);
return e;
}
```

Il meccanismo, con le parole del docente: **tutto quello che viene dopo lo slash viene automaticamente assegnato alla variabile name**. Se chiamo `/employee-rest/ciao` mi ritorna un `Employee` di nome "ciao"; se chiamo `/employee-rest/Pippo` mi ritorna "Pippo"; se chiamo `/employee-rest/Hello` mi ritorna "Hello".

E qui aggiunge **perché** questa cosa si usa così spesso nei REST: perché quella parte di URL viene tipicamente usata per **passare l'ID di una risorsa**. Cioè il caso reale non è `/ciao`, è:

`GET /employee-rest/get/1` → "prendimi l'employee con ID 1"

Il POST: @RequestBody. Per il resto lavorare con gli altri verbi HTTP è simile — "è sufficiente specificare *method* = *POST*". Ma con il POST c'è una differenza importante: il POST **ha bisogno di qualcosa in formato JSON in ingresso**, e quindi bisogna aggiungere l'annotazione **@RequestBody** sul parametro. Quell'annotazione dice a Spring: prendi il corpo della richiesta HTTP, che è JSON, e trasformalo in un oggetto Java di questo tipo.

```
@RequestMapping(value = "/add", method = RequestMethod.POST)
public Employee add(@RequestBody Employee employee) {
    return employee;
}
```

Il modo giusto di tenerlo a mente è come una **simmetria**:

Annotazione	Direzione	Cosa fa
@RequestBody	JSON in ingresso → oggetto Java	deserializza il body della request
ritorno di @RestController	oggetto Java → JSON in uscita	serializza il valore ritornato

Ripasso delle annotazioni più importanti di Spring

Questa lezione è dichiaratamente un **riepilogo** ("le abbiamo già viste in realtà in un altro video") delle annotazioni che si usano più spesso in Spring MVC.

@Controller. La più importante. Messa come intestazione della classe, indica che **quella classe è un controller**.

@RequestMapping. Serve a **mappare un URL su una classe o su un metodo di una classe**. Il punto chiave — che è la cosa che confonde di più chi inizia — è che funziona a **due livelli** e i due livelli si **sommano**:

```
@Controller
```

```

@RequestMapping("/demo")    // ← livello classe: prefisso
public class DemoController {

    @RequestMapping("/welcome") // ← livello metodo
    public String welcome() { ... }
}

```

L'URL che invoca quel metodo è quindi nomeApplicazione/demo/welcome: *"tutti i metodi della classe avranno come prefisso demo"*. Se togliessi il `@RequestMapping` a livello di classe, l'URL sarebbe soltanto /welcome.

Su `@RequestMapping` si possono anche **settare altre informazioni**, in particolare il **tipo di request HTTP** che vogliamo mappare:

```

@RequestMapping(value = "/welcome", method = RequestMethod.GET)

```

Così stiamo dicendo: ogni volta che arriva un URL di questo tipo **con una request HTTP di tipo GET**, richiama questo metodo. Ovviamente si può specificare qualsiasi tipo di request HTTP. Il commento del docente: `@Controller` e `@RequestMapping` sono **le annotazioni che vengono utilizzate più spesso con Spring MVC**.

@PathVariable. Già vista nella lezione 1, qui la riprende: dice che **una parte dell'URL che invoca il metodo dovrà essere associata a una nostra variabile**.

```

@RequestMapping("/hello/{name}")
public String hello(@PathVariable String name) { ... }

```

Invocando /hello/ciao, dietro le quinte quella parte di URL viene inserita come valore dentro name, quindi name = "ciao".

@RequestParam. Qui invece il parametro **non è parte del path**, è un parametro di query — quello che sta dopo il punto interrogativo:

```

@RequestMapping("/api")
public String api(@RequestParam Long id) { ... }

```

chiamato come:

```

/api?id=40    → id = 40

```

E ovviamente se ne possono avere **più di uno**:

```

public String api(@RequestParam Long id, @RequestParam String name) { ... }

```

```

/api?id=40&name=prova    → id = 40, name = "prova"

```

La distinzione che vale la pena fissare, perché è l'unica cosa davvero da ricordare di questa lezione:

	Dove sta il dato	Esempio URL	Uso tipico
@PathVariable	dentro il path	/users/40	identificare una risorsa (l'ID)
@RequestParam	dopo il ?	/users?id=40	filtri, opzioni, ricerche

Le annotazioni "abbreviate": @GetMapping, @PostMapping, ecc. Nelle ultime versioni di Spring, al posto di scrivere per esteso:

```
@RequestMapping(value = "/api", method = RequestMethod.GET)
```

si può scrivere direttamente:

```
@GetMapping("/api")
```

e la stessa cosa vale per tutti gli altri verbi: @PostMapping, @PutMapping, @DeleteMapping, @PatchMapping. Il criterio è *"più conciso, più veloce da scrivere"*: **in base a ogni tipo di request HTTP c'è un'annotazione specifica che in genere finisce con Mapping.**

Nota sull'attualità, perché la lezione le presenta come "novità delle ultime versioni": le annotazioni @GetMapping & co. esistono da **Spring 4.3 (2016)** e oggi sono la **forma normale**. @RequestMapping con method = non è sbagliato, ma su codice nuovo si usa la forma breve; @RequestMapping a livello di **classe** invece resta pienamente in uso, perché lì non c'è un verbo da specificare.

Postman – Introduzione

A cosa serve. Postman (*"il postino"*) serve per **testare i vari servizi REST**. Il docente segnala subito che non è l'unico — cita **SoapUI** come alternativa che *"ha altre funzionalità"* — ma dice che nella quasi totalità dei casi usa Postman. Si scarica gratuitamente da internet, con la motivazione più memorabile del corso: *"noi essendo dei pezzenti prendiamo tutti prodotti gratuiti, i programmatori sono noti per essere pezzenti"*.

Ma perché serve un software apposta? È la domanda che si pone lui stesso, e la risposta è il vero contenuto della lezione. Prende un servizio REST —

`http://localhost:8080/risposte/1`, che ritorna un JSON — e fa notare che **quello lo potrebbe chiamare benissimo dal browser**. Infatti lo apre su Chrome e funziona.

Il problema è che **dal browser, scrivendo un URL nella barra, puoi fare soltanto delle GET**.

Tutte le altre — POST, PUT, DELETE — *"mi verrebbero abbastanza complicate da fare"*. Ed è per questo che serve Postman.

Come si usa, in pratica. Il flusso base è: scrivi l'URL del servizio nella barra in alto, **selezioni alla tua sinistra il tipo di HTTP request**, e fai **Send**. La risposta JSON appare sotto.

Poi mostra i quattro punti in cui si può intervenire su una request, che sono le funzionalità principali di Postman:

Cosa devi fare	Dove si agisce in Postman
Inviare un JSON (POST, PUT)	tab Body → raw → tipo JSON → incolli il JSON → Send
Passare parametri di query	tab Params , oppure scritti a mano nell'URL come <code>?id=2</code>
Aggiungere header	tab Headers

Cosa devi fare

Dove si agisce in Postman

Autenticarsi

tab **Authorization**

Due dettagli utili che emergono dalla demo:

- **Params e URL sono sincronizzati:** man mano che scrivi il parametro nella tabella Params, Postman **te lo inserisce automaticamente nell'URL** in alto, e viceversa. Sono due viste della stessa cosa.
- Sugli **header** fa il collegamento con la sicurezza: *"chi lavora con la sicurezza lo sa, a volte vanno aggiunti degli header nell'intestazione"*. E mostra il caso reale: chiama un servizio POST di login che **gli restituisce un token**, poi va su **Authorization**, sceglie il tipo di autorizzazione (Bearer Token) e ci incolla dentro il token per poter chiamare i metodi protetti. Nella demo il token era scaduto e riceve un *non autorizzato* — ma il meccanismo è quello.

Testare i servizi REST con Postman

Qui si passa alle **funzionalità più avanzate**: creare **test automatici** e poi **automatizzarli**.

Dove si scrivono i test. Nella request si va sul tab **Scripts**, dove ci sono due sezioni:

- **Pre-request** — script eseguito **prima** che parta la richiesta;
- **Post-response** — script eseguito **sulla risposta**.

Attenzione: per creare test con Postman si usa JavaScript. Non Java. La sintassi è quella dei test JavaScript, e l'oggetto pm è *"quello che ci fornisce Postman"*. Il docente si concentra sul **post-response**, che è *"il caso più comune"*, ma precisa che teoricamente si può testare anche nel pre-request.

Il servizio sotto test. È un classico REST CRUD su una `HashMap<Long, User>`, dove `User` è una classe con `id`, `name` ed `email`. I test vengono scritti verbo per verbo.

Test 1 — lo status code (sulla GET)

Il test più semplice: verificare che il servizio risponda **200 OK**.

```
pm.test("Status code is 200", function () {  
    pm.response.to.have.status(200);  
});
```

Test 2 — contenuto del JSON (sulla POST)

Il POST riceve un oggetto `User` in JSON e restituisce l'oggetto creato. Il test prende il JSON di risposta e verifica due cose: che l'**id non sia null** e che il **name sia quello atteso**.

```
const jsonData = pm.response.json();  
  
pm.test("Utente creato con successo", function () {  
    pm.expect(jsonData.id).to.not.eql(null);
```

```
pm.expect(jsonData.name).toEqual("Mario Rossi");
});
```

La parte didatticamente più utile è quando il test fallisce. Invia un POST con nome Antonio Rossi mentre nel test si aspettava Mario Rossi → compare **Test failed**. Il ragionamento che fa per capirlo: *"lui mi ritorna l'oggetto user, quindi quello che gli passo lui mi ritorna"* — cioè il POST fa da eco, e se in input ho messo Antonio, il test deve aspettarsi Antonio. Corregge l'atteso e il test passa. Lo stesso identico errore lo ripete sulla PUT: aggiorna con Mario aggiornato ma il test si aspettava Mario Rossi, fallisce, corregge, passa.

Il punto generale, che vale ben oltre Postman: quando un test fallisce, il colpevole può essere **il codice o il test stesso**, e la prima cosa da fare è chiedersi quale dei due sta dicendo la verità.

Test 3 — presenza dei campi (sulla GET by id)

Qui non ci interessa il valore, solo che **certe proprietà esistano** nel JSON:

```
pm.test("La risposta contiene id, name ed email", function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData).to.have.property("id");
  pm.expect(jsonData).to.have.property("name");
  pm.expect(jsonData).to.have.property("email");
});
```

Anche qui c'è un fallimento istruttivo: chiama l'utente 4, ottiene **404 Not Found** e tutti i test falliscono — semplicemente perché *"l'ultimo utente che ho inserito aveva ID forse due"*, quindi l'utente 4 non esiste.

Test 4 — testare l'errore atteso (sulla DELETE)

La DELETE è **void**, quindi non ritorna nulla e non c'è un JSON da controllare. Ma si può testare qualcosa di più interessante: **dopo aver cancellato, una GET sullo stesso id deve dare 404**.

```
pm.test("Utente eliminato: ora ritorna 404", function () {
  pm.response.to.have.status(404);
});
```

E il 404 arriva perché lato Java il servizio è scritto così: se nella map non c'è l'elemento con quella chiave, viene lanciata una `UserNotFoundException` che si traduce in `HttpStatus.NOT_FOUND`.

Test 5 — performance e header

```
pm.test("Risposta sotto i 300ms", function () {
  pm.expect(pm.response.responseTime).to.be.below(300);
});
```

```
pm.test("Content-Type è application/json", function () {  
  pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json");  
});
```

Nota importante che dice esplicitamente: si possono applicare più test al singolo servizio REST. Non è uno per request.

Test 6 — iterare su una lista

Sulla GET che ritorna l'elenco utenti: la lista non deve essere vuota, e **ogni** elemento deve avere le sue proprietà.

```
pm.test("La lista non è vuota e ogni utente ha id ed email", function () {  
  const users = pm.response.json();  
  pm.expect(users.length).to.not.eql(0);  
  users.forEach(function (u) {  
    pm.expect(u).to.have.property("id");  
    pm.expect(u).to.have.property("email");  
  });  
});
```

Il commento del docente su tutto questo: *"possiamo definire una marea di test, il limite è la fantasia"*.

Eseguire tutti i test insieme: Run Collection

Fin qui abbiamo test sparsi su singole request. Per lanciarli tutti insieme — una specie di **test suite** — si va sulla **collection**, **tasto destro** → **Run collection**, poi **Run**. Postman esegue in sequenza tutte le request e mostra il **riepilogo di tutti i test** eseguiti, con quali sono passati e quali no.

E l'ultima cosa che segnala, importante perché è il ponte verso la CI: c'è anche la possibilità di **schedulare** l'esecuzione, oppure di **lanciarla da linea di comando** — *"ogni tot, la notte, vogliamo lanciare questi test"*.

Nel mondo reale il pezzo da linea di comando si chiama **Newman**, il runner CLI di Postman: `newman run collection.json`. È quello che permette di infilare la collection dentro una pipeline Jenkins/GitHub Actions.

Chiude rimandando alla **documentazione ufficiale** (`learning.postman.com`, sezione test scripts), dove c'è l'elenco degli oggetti JavaScript definiti dentro Postman, delle variabili e dei metodi principali.

La collection allegata

Nello zip c'è `Postman_Automated_Tests_Demo.postman_collection.json`: una collection minima, in **schema v2.1.0**, con **una sola request** — GET

`https://jsonplaceholder.typicode.com/users/1` — e tre test nell'evento `test`, che sono esattamente i tre pattern base visti sopra:

```
pm.test("Status code is 200", function () {
  pm.response.to.have.status(200);
});

pm.test("Response has user id", function () {
  let jsonData = pm.response.json();
  pm.expect(jsonData).to.have.property("id");
});

pm.test("User name is Leanne Graham", function () {
  let jsonData = pm.response.json();
  pm.expect(jsonData.name).to.eql("Leanne Graham");
});
```

È autonoma (punta a `jsonplaceholder.typicode.com`, un'API pubblica di prova), quindi **si può importare in Postman e lanciare subito**, senza avviare nessuna applicazione Spring: Import → seleziona il file → Run collection.

Lombok

Cos'è. Lombok è una **libreria che si può usare a prescindere da Spring** — *"vive di vita propria anche al di fuori di Spring"* — ma dentro Spring viene usata tantissimo dai programmatori.

Il problema, mostrato prima della soluzione. Crea una normalissima classe `Impiegato` con due campi:

```
public class Impiegato {
  private String nome;
  private String cognome;
}
```

Poi elenca tutto quello che una *"classe ben fatta"* deve avere, essendo gli attributi privati, e lo genera con i wizard di Eclipse:

- i metodi **getter e setter** (Source → Generate Getters and Setters);
- i **costruttori** (Generate Constructor using Fields);
- **`hashCode()` e `equals()`**;
- **`toString()`**.

Il risultato funziona, ci ha messo poco grazie ai wizard — ma la classe è diventata **lunghissima**, e tutto quel codice è **codice standard**, sempre uguale, che non dice nulla sul dominio.

La soluzione. Prima si aggiunge la dipendenza nel `pom.xml`:

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
</dependency>
```

Poi crea una seconda classe, `Impiegato2`, con **gli stessi identici campi**, per poter fare il confronto:

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Impiegato2 {
  private String nome;
  private String cognome;
}
```

Le tre annotazioni, con il significato che dà lui:

Annotazione	Cosa genera automaticamente
@NoArgsConstructor	il costruttore senza argomenti
@AllArgsConstructor	un costruttore con tutti gli attributi della classe
@Data	i metodi get e set , <code>hashCode()</code> , <code>equals()</code> (e <code>toString()</code>)

La conclusione: *"non c'è proprio paragone tra questa classe e quest'altra, perché sono la stessa identica cosa, solo che questa è molto più lunga"*. Con tre annotazioni abbiamo creato tutto quello che serve.

Rimanda infine al sito ufficiale — `projectlombok.org`, sezione **Feature Overview** — perché *"ovviamente ci sono molte più annotazioni"* e molti più modi di configurarlo.

Due cose che la lezione non dice e che vale la pena sapere. **Primo:** Lombok funziona come *annotation processor*, cioè genera il codice **in fase di compilazione**; nell'IDE va installato/abilitato il plugin, altrimenti l'editor segna errori rossi anche se Maven compila. In Spring Boot la dipendenza si mette con `<optional>true</optional>` (lo starter parent ne gestisce già la versione, per questo nel pom della lezione non c'è il tag `<version>`). **Secondo:** `@Data` su una **entity JPA** è una cattiva idea, perché genera `equals/hashCode` su *tutti* i campi, incluse le relazioni lazy — sulle entity si preferiscono `@Getter`, `@Setter` e `@ToString(exclude = ...)` separati. E dal Java 17 in poi, per i DTO immutabili spesso al posto di Lombok si usa direttamente un **record**.